

The Adapter Pattern

Human Computer Interaction Research
University of Nevada, Reno

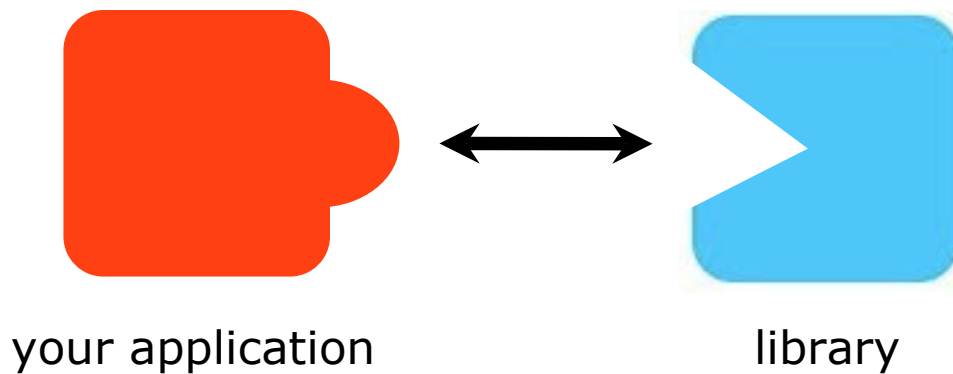


Adapter

- Behavioral Patterns ~ communication between objects
 - » strategy
 - » decorator
 - » command
- Creational Patterns ~ object creation
 - » factory method
 - » abstract factory
 - » singleton
- Structural Patterns ~ object modification
 - » decorator
 - » adapter

Problem

“Interface Incompatibility, and you cannot (don’t want to) change the interface of the library you want to use”



round and square pegs



Peg & Square classes

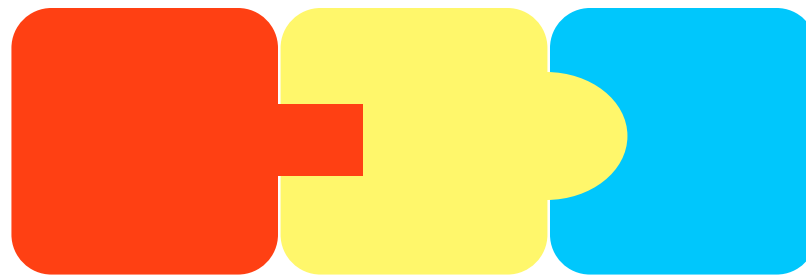
```
public class SquarePeg {  
    public void insertIntoSquareHole(String str) {  
        System.out.println("insert into Square(): " + str);  
    }  
}
```

```
public class RoundPeg {  
    public void insertIntoRoundHole(String str) {  
        System.out.println("insert into Round(): " + str);  
    }  
}
```

If a client only understands the SquarePeg interface for inserting pegs using the insertIntoSquareHole() method, how can it insert round pegs?

Adapter / Wrapper

The Adapter Pattern converts the interface of a class into another interface the client expects.

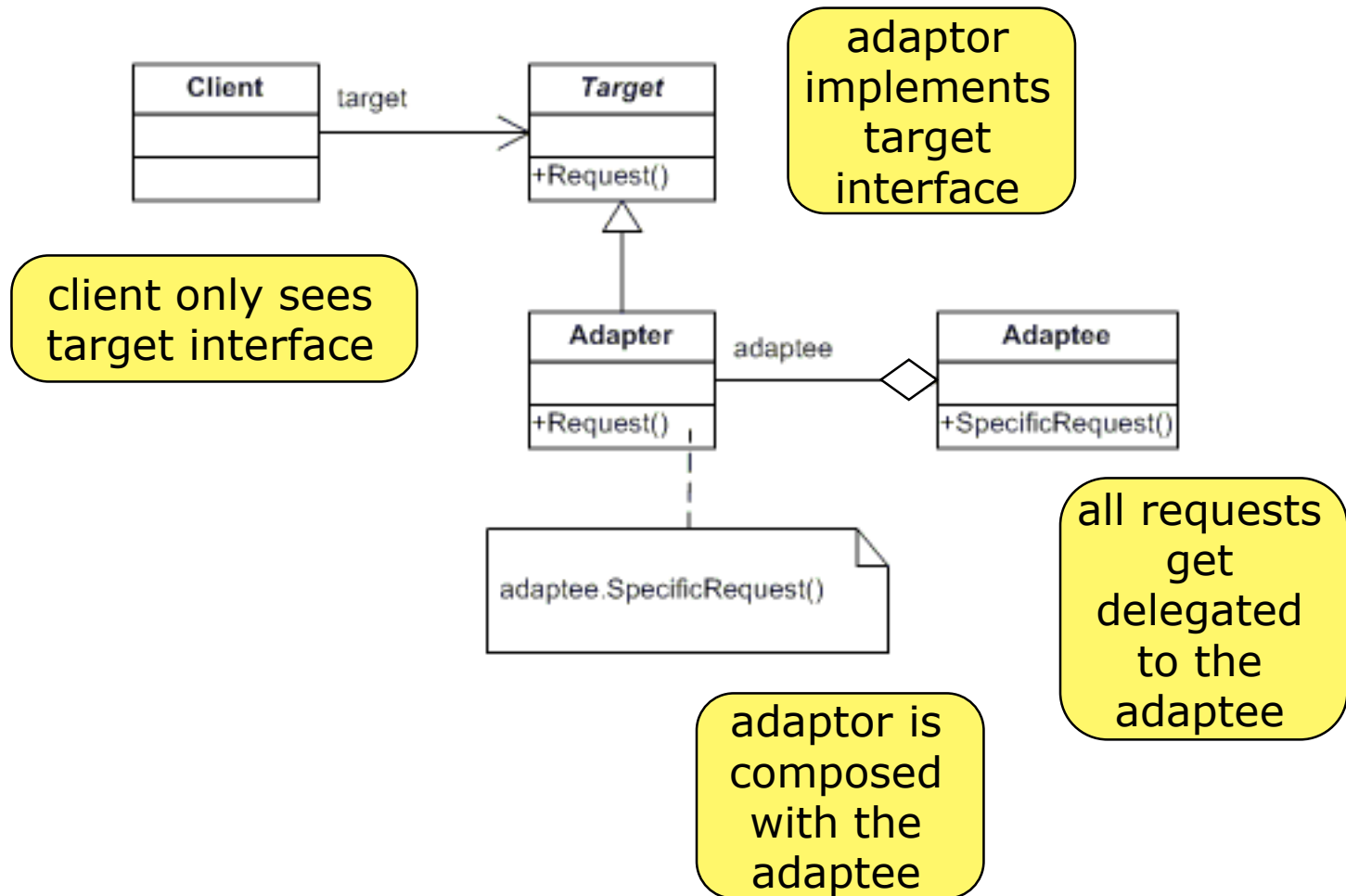


square adapter round

?

`insertIntoSquareHole()` $\longleftrightarrow$ `insertIntoRoundHole()`

Object Adapter



PegAdapter

```
public class PegAdapter extends SquarePeg {  
    private RoundPeg roundPeg;  
  
    public PegAdapter(RoundPeg peg) {  
        this.roundPeg = peg;  
    }  
  
    public void insertIntoSquareHole(String str) {  
        // override  
        roundPeg.insertIntoRoundHole(str);  
    }  
}
```



map insertIntoSquareHole()
to insertIntoRoundHole()

Main/Output

```
public class TestPegs {  
    public static void main(String[] args) {  
        RoundPeg roundPeg = new RoundPeg();  
        SquarePeg squarePeg = new SquarePeg();  
        roundPeg.insertIntoRoundHole("round peg");  
        squarePeg.insertIntoSquareHole("square peg");  
        PegAdapter adapter = new PegAdapter(roundPeg);  
        adapter.insertIntoSquareHole("square peg");  
    }  
}
```

```
insert into Round(): round peg  
insert into Square(): square peg  
insert into Round(): square peg
```

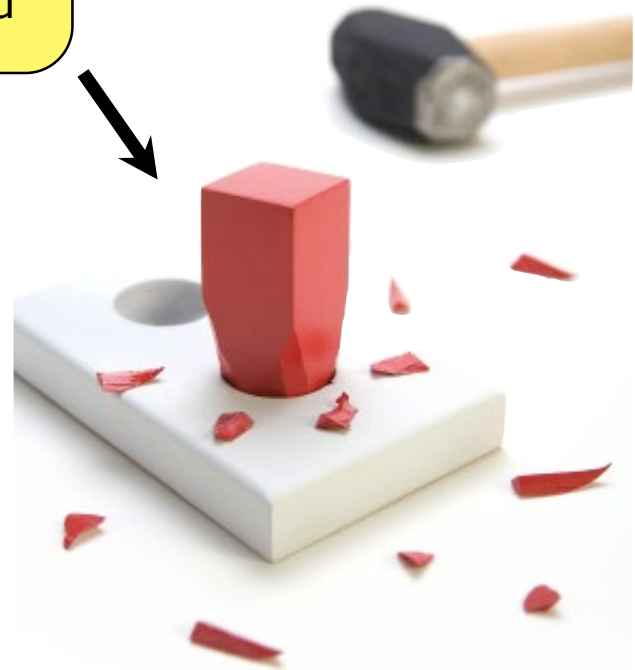
using interfaces

peg adapter adapts a square peg to a round peg.

can we have an adapter that acts as a square peg or a round peg?

Java has no multiple inheritance

but an adapter can implement 2 interfaces



Refactor using Interfaces

```
public interface IRound {  
    public void insertIntoRoundHole(String str);  
}  
  
public interface ISquare {  
    public void insertIntoSquareHole(String str);  
}  
  
public class SquarePeg implements ISquare {  
    public void insertIntoSquareHole(String str) {  
        System.out.println("insert into Square(): " + str);  
    }  
}  
  
public class RoundPeg implements IRound {  
    public void insertIntoRoundHole(String str) {  
        System.out.println("insert into Round(): " + str);  
    }  
}
```

2-way adapter

```
public class PegAdapter implements ISquarePeg, IRoundPeg {  
    private RoundPeg roundPeg;  
    private SquarePeg squarePeg;  
  
    public PegAdapter(RoundPeg peg) {  
        this.roundPeg = peg;  
    }  
    public PegAdapter(SquarePeg peg) {  
        this.squarePeg = peg;  
    }  
    public void insertIntoSquareHole(String str) {  
        roundPeg.insertIntoRoundHole(str);  
    }  
    public void insertIntoRoundHole(String str) {  
        squarePeg.insertIntoSquareHole(str);  
    }  
}
```

Test

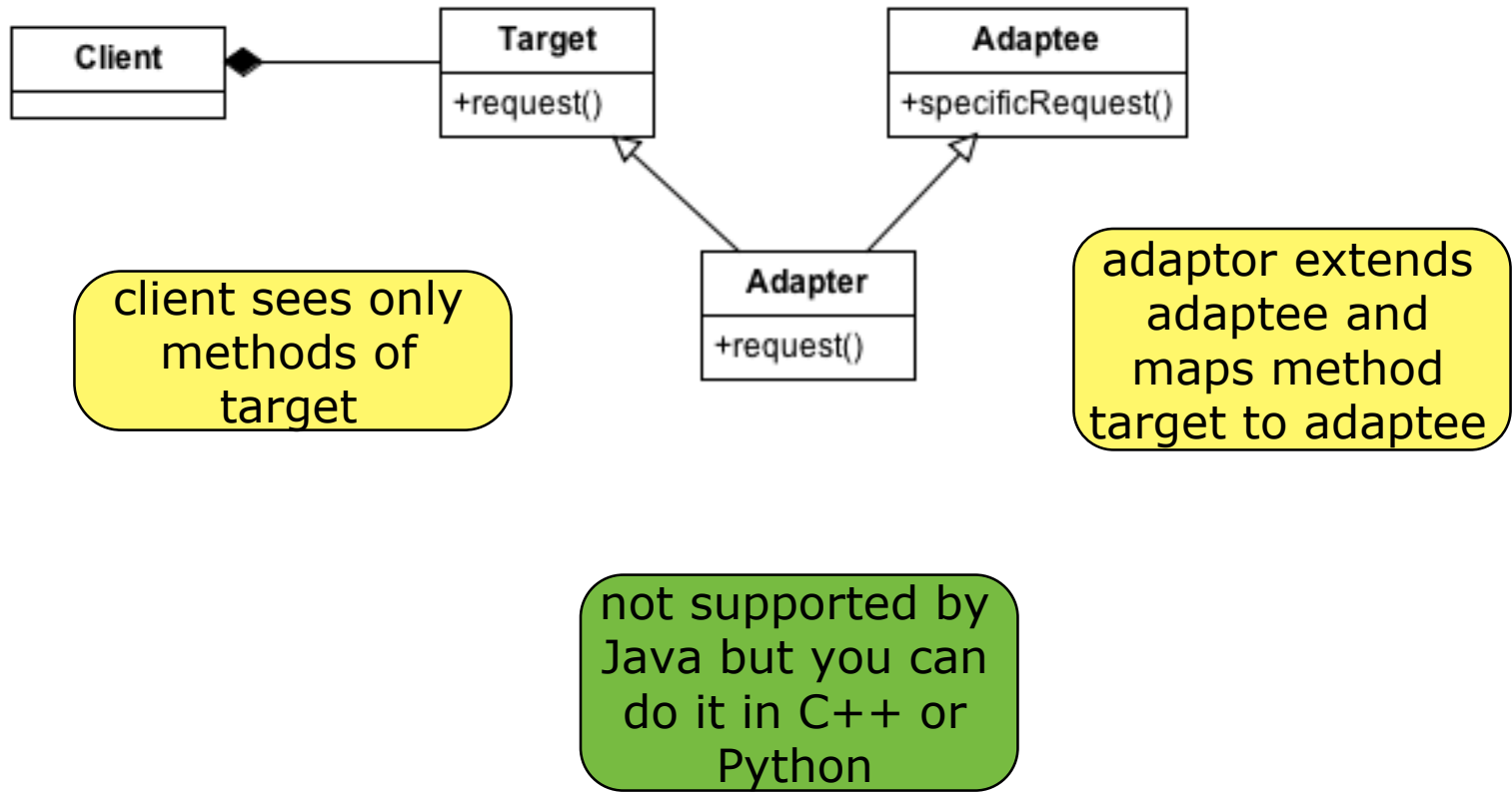
```
public class TestPegs {  
    public static void main(String[] args) {  
        RoundPeg roundPeg = new RoundPeg();  
        SquarePeg squarePeg = new SquarePeg();  
        roundPeg.insertIntoRoundHole("round peg");  
        squarePeg.insertIntoSquareHole("square peg");  
  
        IRoundPeg roundToSquare = new PegAdapter(squarePeg);  
        roundToSquare.insertIntoRoundHole("round peg");  
        ISquarePeg squareToRound = new PegAdapter(roundPeg);  
        squareToRound.insertIntoSquareHole("square peg");  
    }  
}
```

```
insert into Round(): round peg  
insert into Square(): square peg  
insert into Square(): round peg  
insert into Round(): square peg
```

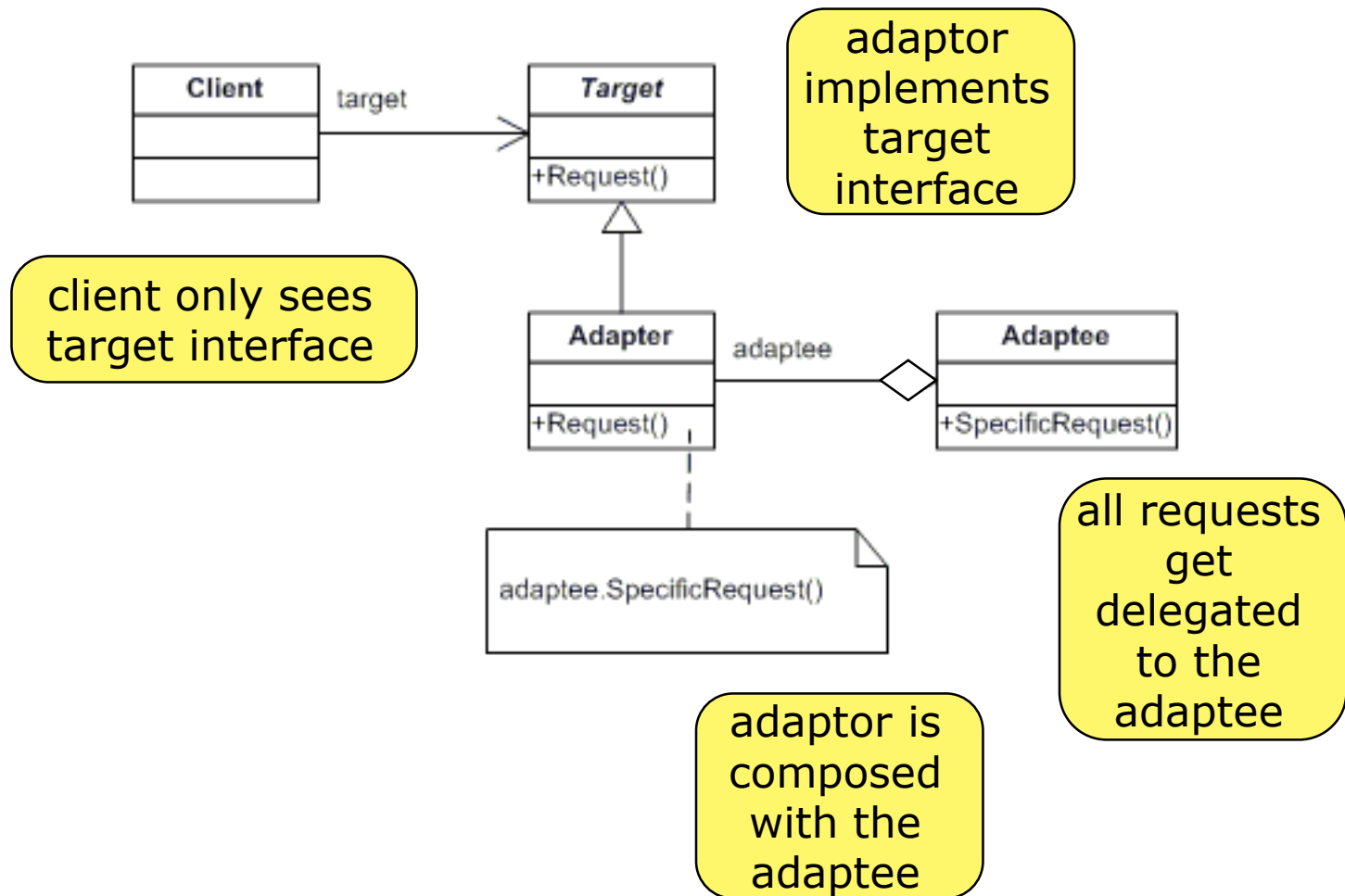
two types of adapters



Class Adapter – is a



Object Adapter – has a



Tradeoffs

Object Adapter

can adapt any
subclasses of
adaptee

has to reimplement
adaptee

adaptee + adapter

new behavior works
with all subclasses

Class Adapter

committed to one
adaptee class

just override
methods

class adapter

new behavior works
only with adaptee

doesn't work in java

projector

desk

Group 1

Group 2

Group 3

Group 4

Group 5

Group 6

Group 7

Group 8

Group 9

Group 10

Group 11

Group 12

pillar

Group 13

Group 14

Group 15

Group 16

Group 17

Group 18

Group 19

Group 20

Group 21

Group 22

Group 23

Group 24

Group 25

Group 26

Group 27

Group 28

Group 29

pillar

skateboards

door